YourNameMart — Project Requirements & Execution Specification

Java Servlets · JDBC · Apache Tomcat — Anna University R2025, Semester 3

Checkpoint window: Jul 27 – Oct 10, 2026 · **Builder:** solo

> Replace `YourNameMart` (and `yournamemart` in package names, file paths, and JDBC URLs below) with a name of your own — for example, if your name is Aditya, the project becomes `AdityaMart`, package `com.aditya.adityamart`. Apply this consistently across the codebase, README, and deployed project title.

This document specifies the requirements, architecture, timeline, and execution process for this project. All requirements are mandatory unless explicitly marked optional. Sections are to be executed in the order presented.

---

1. Project Definition & Feature Requirements

Project type: Multi-seller e-commerce marketplace web application ("YourNameMart").

Description: Sellers list products. Buyers browse, search, add to cart, and purchase. An admin manages users, orders, and listings.

Scope constraints:

- No real-time infrastructure (WebSockets, live location tracking, external mapping APIs).
- No third-party payment gateway integration. Checkout implements a mock payment confirmation step.

Mandatory feature requirements:

ID	Requirement
F1	User registration and login with two roles: Buyer, Seller. Admin role assigned via a seed account; no separate admin signup flow.
F2	Seller: create, edit, delete product listings (name, description, price, stock quantity, category, image URL).
F3	Buyer: browse and search/filter products by category and keyword.
F4	Cart: add, update, remove items; display running total.
F5	Checkout: place an order from cart contents via mock payment confirmation.
F6	Order history: buyer view of past orders and status; seller view of incoming orders for their products.
F7	Admin: view all users and orders; moderate/remove listings.
F8	Product reviews and star ratings on completed orders.

Optional feature requirements (implement only after F1–F8 are complete and tested):

ID	Requirement
O1	Wishlist / save-for-later.
O2	Order status workflow: Pending → Confirmed → Shipped → Delivered.
O3	Seller sales dashboard (order counts, revenue totals).
O4	AI chatbot — mandatory deliverable for the Final Review (Section 11), sequenced as optional only relative to F1–F8.

Action item — due Jul 27: Initialize the GitHub repository.

---

2. System Architecture Specification

Architecture pattern: Layered MVC over Servlets (Front Controller pattern).

```
Browser (HTML/CSS/vanilla JS + AJAX fetch)
        |  HTTP request
        v
Filter layer      -> AuthFilter (session check), EncodingFilter, CORSFilter
(if needed)
        v
Front Controller  -> single DispatcherServlet OR per-resource Servlets
(ProductServlet, OrderServlet...)
        v
Service layer     -> business logic, validation (no JDBC here)
        v
DAO layer         -> ProductDAO, OrderDAO ... — ALL SQL lives here,
PreparedStatement only
        v
Connection Pool   -> HikariCP (via a ServletContextListener at startup)
        v
H2 Database (server mode)
```

Package structure:

```
com.yourname.yournamemart
 |-- controller     (Servlets — thin, no SQL, no business logic)
 |-- service        (business rules, orchestration)
 |-- dao            (interfaces + JDBC implementations)
 |-- model          (POJOs / entities)
 |-- dto            (request/response shapes for JSON endpoints)
 |-- filter         (auth, logging, encoding)
 |-- listener       (DataSource init/teardown)
 |-- util           (PasswordUtil, ValidationUtil, JsonUtil)
 `-- exception      (custom checked exceptions, global error handling)
```

Mandatory engineering rules:

1. All SQL statements use `PreparedStatement`. No string-concatenated queries, under any circumstance.
2. Passwords hashed with bcrypt (jBCrypt). No plaintext storage. No MD5/SHA1-only hashing.
3. Session management via `HttpSession`. Regenerate session ID on login. Set an explicit session timeout.
4. Escape all user-supplied output before rendering (JSTL `<c:out>` or `fn:escapeXml`).
5. Connection pool lifecycle owned by a single `ServletContextListener`. No `DriverManager.getConnection()` calls outside this listener.
6. Try-with-resources required for every `Connection`, `PreparedStatement`, `ResultSet`.

---

3. Technology Stack Specification

Component	Specification
JDK	17 (LTS)
Servlet container	Tomcat 9.0.x — Servlet API (`javax.servlet.*`). Confirm with faculty guide before starting development.
Build tool	Maven
Database	H2 — server mode for the deployed application, embedded mode for local development and automated tests
JDBC driver	(`com.h2database:h2`) — driver class (`org.h2.Driver`)
Connection pooling	HikariCP, initialized via (`ServletContextListener`)
View layer	JSP + JSTL for server-rendered pages; vanilla JS + (`fetch()`) for AJAX endpoints
JSON serialization	Gson
Password hashing	jBCrypt
Testing	JUnit 5 + Mockito. DAO tests execute against an embedded H2 instance (`jdbc:h2:mem:test`)
Logging	SLF4J + Logback
CI	GitHub Actions — (`mvn -B test`) on every push

4. Database Design Specification

Minimum schema:

```
users(id PK, name, email UNIQUE, password_hash, role
ENUM('BUYER','SELLER','ADMIN'), created_at)
products(id PK, seller_id FK->users, name, description, price, stock_qty,
category, created_at)
orders(id PK, buyer_id FK->users, status
ENUM('PENDING','CONFIRMED','SHIPPED','DELIVERED','CANCELLED'),
total_amount, created_at)
order_items(id PK, order_id FK->orders, product_id FK->products, quantity,
unit_price)
cart_items(id PK, user_id FK->users, product_id FK->products, quantity)
reviews(id PK, product_id FK->products, user_id FK->users, rating, comment,
created_at)
```

Mandatory design rules:

1. Every foreign key indexed.
2. `email` column unique-constrained.
3. Monetary values stored as `DECIMAL(10,2)`. `FLOAT` prohibited for currency fields.
4. `created_at` timestamp present on every table.
5. `schema.sql` and `seed.sql` checked into the repository.

---

5. Required Design Diagrams

Three diagrams are mandatory deliverables. Produce each using a diagramming tool of your choice (draw.io, Lucidchart, PlantUML, or hand-drawn and scanned). Each must reflect this specification's actual content — not a generic template.

ID	Diagram	Must depict	Source it derives from
D1	ER Diagram	All entities and relationships in the schema	Section 4 (Database Design Specification)
D2	Use Case Diagram	Actors (Buyer, Seller, Admin) and their use cases	Section 1 (feature requirements F1–F8)
D3	Sequence Diagram	The place-order flow: Browser → Servlet → Service → DAO → Database, including the response path	Section 2 (System Architecture Specification)

Submission requirement: All three diagrams included in the final report (Section 7) and referenced in the README. D2 must additionally accompany the Checkpoint 1 problem statement submission.

6. Execution Timeline & Milestones

Window	Dates	Required Output	Checkpoint
Kickoff	Jul 24 – Jul 27	Problem statement locked, repository initialized, Tomcat/Maven skeleton running, DB schema v1	Problem statement due Jul 27
Week 1	Jul 27 – Aug 2	Authentication (register/login/session), base DAO layer, project skeleton deployed locally	—
Week 2	Aug 3 – Aug 9	Core flow: browse → add to cart → place order	—
—	Aug 10	Working MVP demo	MVP Review
Week 3	Aug 10 – Aug 16	Review feedback addressed; seller dashboard (listing management) started	—
Week 4	Aug 17 – Aug 23	Seller dashboard complete; admin panel started	—
Week 5	Aug 24 – Aug 30	Search/filter, order status workflow	—
Week 6	Aug 31 – Sep 6	Reviews/ratings, edge case handling, input validation	—
Week 7	Sep 7 – Sep 13	Security requirements (Section 9) implemented; unit and DAO test coverage completed	—
Week 8	Sep 14 – Sep 20	Deployment finalized; live URL smoke-tested	Deploy link due Sep 21
—	Sep 21	Full build demo on live cloud link	2nd Review

Window	Dates	Required Output	Checkpoint
Week 9	Sep 21 – Sep 27	AI chatbot integration — backend proxy servlet + widget	—
Week 10	Sep 28 – Oct 4	Chatbot refinement, UI polish, README/architecture documentation	—
Week 11	Oct 5 – Oct 10	Final regression pass, report, slide deck, rehearsed demo	Final Review Oct 10

7. Checkpoint Deliverable Requirements

MVP Review — Aug 10:

- [ ] One complete end-to-end user journey functional
- [ ] DB schema implemented and populated with seed data
- [ ] Authentication functional (register/login/session)
- [ ] Repository contains a minimum of 6 commits (3 commits/week x 2 weeks, per Section 8)
- [ ] 2-slide summary: problem statement, architecture sketch, completed vs. planned work

Full Build + Deploy — Sep 21:

- [ ] All F1–F8 requirements implemented
- [ ] Live cloud URL functional and verified
- [ ] Security checklist (Section 9) completed
- [ ] Test suite passing in CI
- [ ] README complete: setup instructions, architecture, screenshots

Final Review — Oct 10:

- [ ] AI chatbot integrated and functional

- [ ] Full regression pass completed — no functionality present at Sep 21 broken
- [ ] Final report: architecture, ER diagram, technical decisions, known limitations
- [ ] Slide deck, rehearsed demo script, backup demo video
- [ ] Repository commit history shows a minimum of 3 commits per week maintained across the full checkpoint window

---

8. Version Control & CI Requirements

1. Minimum 3 commits per week, every week, Jul 27 – Oct 10 (minimum 33 commits over the checkpoint window).
2. Branch model: `main` always deployable; `feature/<name>` branches per feature; merge via self-reviewed pull request.
3. Commit message format: conventional commits — `feat:`, `fix:`, `test:`, `docs:`.
4. CI pipeline required. `.github/workflows/build.yml`:

```yaml
name: build-and-test
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin' }
      - run: mvn -B clean verify
```

5. `README.md` required at repository root: problem statement, architecture diagram, tech stack table, setup instructions, deployed link, screenshots.

---

9. Testing & Validation Requirements

Layer	Test scope	Tooling
DAO	CRUD operations against schema	JUnit 5 against `jdbc:h2:mem:test;DB_CLOSE_DELAY=-1`, initialized from `schema.sql` per test run
Service	Business rule validation	JUnit 5 + Mockito (DAO mocked)
Servlet	Request/response behavior, status codes, session handling	Mockito `HttpServletRequest`/`Response` mocks, or Postman collections
Security	SQL injection, XSS payloads, auth bypass on protected routes	Manual checklist below
End-to-end	register → browse → order → review	Documented manual test-case sheet
Load	Concurrent user handling	Apache JMeter or `ab` — minimum 10 concurrent users, 60 seconds

Security checklist — complete before Sep 21:

- [ ] Every query parameterized. Verify: `grep -rn "Statement)" src/`, confirm each result is `PreparedStatement`.
- [ ] Passwords bcrypt-hashed, never logged.
- [ ] All protected servlets enforce session checks via `AuthFilter`.
- [ ] User-supplied input escaped before rendering.
- [ ] File upload (if implemented) validates type and size; never trusts client-supplied filename.

- [ ] Error pages do not expose stack traces (`web.xml` custom error pages).
- [ ] Database credentials excluded from version control (`config.properties` in `.gitignore`).

---

10. Deployment Specification

Deployment requirements (mandatory, platform-agnostic):

1. Publicly accessible URL, reachable for the duration of the checkpoint window (Jul 27 – Oct 10).
2. Application runs on a Servlet container (Tomcat 9.0.x) serving the built `.war`.
3. Database (H2, or equivalent per Section 3) reachable by the deployed application, with data persisting across restarts.
4. Availability guaranteed during scheduled review sessions (Aug 10, Sep 21, Oct 10).

Platform selection: Any platform meeting the requirements above is acceptable — a self-managed cloud VM (free-tier or otherwise), a PaaS with Java/Tomcat or Docker support, or an institution-provided server. Selection is left to the student. Verify current free-tier terms and capacity before committing, as these change over time.

Reference setup (applicable if self-hosting on a Linux VM):

1. Provision the VM — **Week 1**.
2. Install JDK: `apt install openjdk-17-jdk`.
3. Install Tomcat 9.0.x; configure as a systemd service.
4. Open port 8080, or configure a reverse proxy (e.g., Nginx) on port 80/443 with a TLS certificate.
5. Configure H2 in server mode as a systemd service: `java -cp h2*.jar org.h2.tools.Server -tcp -tcpAllowOthers -baseDir /opt/yournamemart/data`. Execute `schema.sql` against it once.
6. Build: `mvn clean package`. Deploy the resulting `.war` to Tomcat's `webapps/` folder. JDBC URL: `jdbc:h2:tcp://localhost:9092/./data/yournamemart`.

7. Verify every core user journey against the live URL prior to each review.
8. Configure uptime monitoring (e.g., UptimeRobot, 5-minute interval). Back up the H2 data file (`.mv.db`) after each review.

Backup requirement: Record a 2–3 minute screen-capture demonstration prior to each review, for use if the live deployment is inaccessible during evaluation.

---

11. AI Feature Specification (Phase 3)

Required architecture:

```
Chat widget (JS, floating button + panel)
        |  fetch('/api/chat', POST {message})
        v
ChatServlet  (validates input, enforces per-session rate limit)
        v
ChatService  -> calls an LLM API server-side
        v
Returns JSON { reply: "..." } -> widget renders it
```

Requirements:

1. API key stored server-side only (properties file or environment variable). Must not appear in client-side code.
2. Chatbot scope restricted to product/listing domain queries.
3. API call wrapped in try/catch. On failure, return a static degraded response instead of an error page.
4. Minimum functional scope for Oct 10: correct responses to 5–10 FAQ-style domain questions, with functioning UI integration.

---

12. Coding Standards

Rule	Standard
Naming	Classes `PascalCase`, methods/variables `camelCase`, constants `UPPER_SNAKE_CASE`, packages lowercase, no abbreviations
Documentation	Javadoc required on every public class and method in `service`/`dao` layers
Static analysis	Checkstyle and SpotBugs configured as Maven plugins, run in CI
SOLID	Servlets restricted to HTTP orchestration. Business rules confined to `service`. `Service` classes depend on DAO interfaces, not concrete DAO classes
Design patterns required	DAO (data access abstraction), Front Controller (central dispatch), Singleton (connection pool via `ServletContextListener`), Factory (DAO/provider instantiation), Strategy (swappable notification/payment-mock channels), Builder (complex DTO construction). Document pattern usage in the final report

---

13. API Contract Standards

1. All JSON endpoints versioned under `/api/v1/...`.
2. Response envelope, fixed format:

```json
{ "success": true,  "data": { ... }, "error": null }
{ "success": false, "data": null,    "error": { "code": "VALIDATION_ERROR", "
```

3. HTTP status codes: 200/201 success, 400 bad input, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 500 server error. Do not return HTTP 200

for error responses.

4. DTOs are separate classes from entities. `UserResponseDTO` must not contain `passwordHash`.
5. Validation occurs at the top of the service method. Return HTTP 400 with a field-level error before any DAO call.

---

14. Database Migration Requirements

1. Every schema change is a new, numbered, checked-in file: `db/migrations/V1__init_schema.sql`, `V2__add_reviews_table.sql`, `V3__add_index_orders_status.sql`. Do not hand-edit a live table.
2. Optional: Flyway, applied on application startup, to auto-run new migrations.
3. Seed/demo data maintained in a script separate from schema migrations.

---

15. Change Management Workflow

1. **Intake:** Every feature or bug originates as a GitHub Issue using a template:
   - Feature/Change Request: description, motivation, acceptance criteria, affected layers.
   - Bug report: steps to reproduce, expected vs. actual, environment.
2. **Impact analysis:** 3–4 lines on the issue prior to implementation — DB migration required? API response shape changed? Additive or shared-code change?
3. **Branch and PR:** `feature/<name>` branch. PR description states what changed, why, and how it was tested. Checklist: tests added, docs updated, migration included if schema changed.
4. **Versioning:** Semver tags — `v0.1.0` (MVP) → `v0.2.0` … `v1.0.0` (Sep 21 build) → `v1.1.0` (AI chatbot).
5. **CHANGELOG.md:** One line per release, updated as part of the PR that ships it.

---

16. Sprint Execution Cycle

1. Timeline (Section 6) executed as five ~2-week sprints.
2. Each sprint: sprint planning (select 3–5 issues from backlog) → mid-sprint check-in at day 7 → sprint review against Definition of Done (Section 19) → retro entry in `RETRO.md` (one line: what worked, what didn't, one change for the next sprint).
3. Work tracked on a GitHub Projects Kanban board: `Backlog → In Progress → In Review → Done`.

---

17. AI Integration Architecture Requirements

1. AI provider accessed through an interface, not a hardcoded implementation:

```java
public interface ChatProvider {
    String getReply(String userMessage, String context);
}
// GeminiChatProvider implements ChatProvider { ... real API call ... }
// MockChatProvider  implements ChatProvider { ... canned FAQ answers, no net
```

2. Implementation selected via config flag (`ai.chatbot.provider=gemini|mock`).
3. Guardrails required: per-session rate limit (10 messages/minute), input length cap, timeout on outbound API call, fixed server-side prompt template.
4. Repeated identical questions cached in-memory per session.

---

18. Observability & Environment Requirements

1. `GET /api/v1/health` returns `{ "status": "UP", "db": "UP" }`, verifying database connectivity.
2. Structured logging via Logback: request ID generated per incoming request in a `Filter`, attached to the SLF4J MDC.

3. `.env.example` committed; actual `.env` excluded via `.gitignore`, documenting every required config value.
4. `CONTRIBUTING.md`: exact steps from `git clone` to a running local instance.

---

19. Definition of Done

Applies to every feature prior to merge:

- [ ] Compiles with no Checkstyle/SpotBugs major warnings
- [ ] Unit/DAO tests written and passing
- [ ] Code self-reviewed prior to merge
- [ ] Migration script included if schema changed
- [ ] Verified against the deployed URL, not only localhost
- [ ] README/API documentation updated if behavior changed
- [ ] Merged to `main` only when CI is green

This specification is binding for the checkpoint window Jul 27 – Oct 10, 2026.